

26F Stat TA Session W5

More Distributions

- Recall that we learn how to generate random numbers with `runif()` (uniform distribution)
- As the lecture continues, we'll be familiar with more common distributions. e.g., **binomial distribution** and **normal distribution**

Binomial Distribution

- Before we are introduced to Binomial distribution, we first recall the concept of **Bernoulli distribution**
- A random variable following Bernoulli distribution assigns 1 if the trial succeeds; 0 if the trial fails



$$X \sim \text{Bernoulli}(p), X = \begin{cases} 1, & \text{if the trial succeeds} \\ 0, & \text{if the trial fails} \end{cases}$$

- Binomial distribution** is a close related to the Bernoulli distribution. It counts **the number of successful trials** among a total n independent trails
- Now let's draw random samples from binomial distribution using `rbinom()`

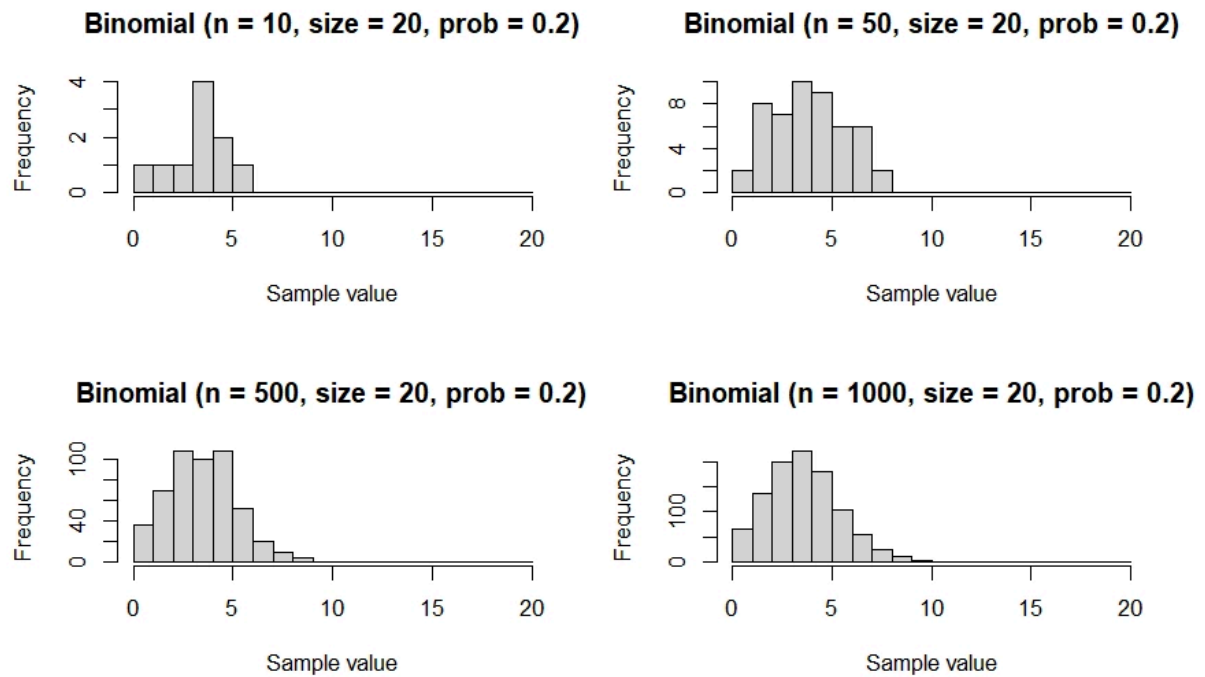


Figure: Random Samples drawn from Binomial(20, 0.2) in different sizes

```
rdnum <- rbinom(n = 30, size = 20, prob = 0.5) # n = number of observations # size =
number of trials
```

Normal Distribution

- Normal distribution models data that are centered around the mean, with most values close to the mean and fewer values farther away

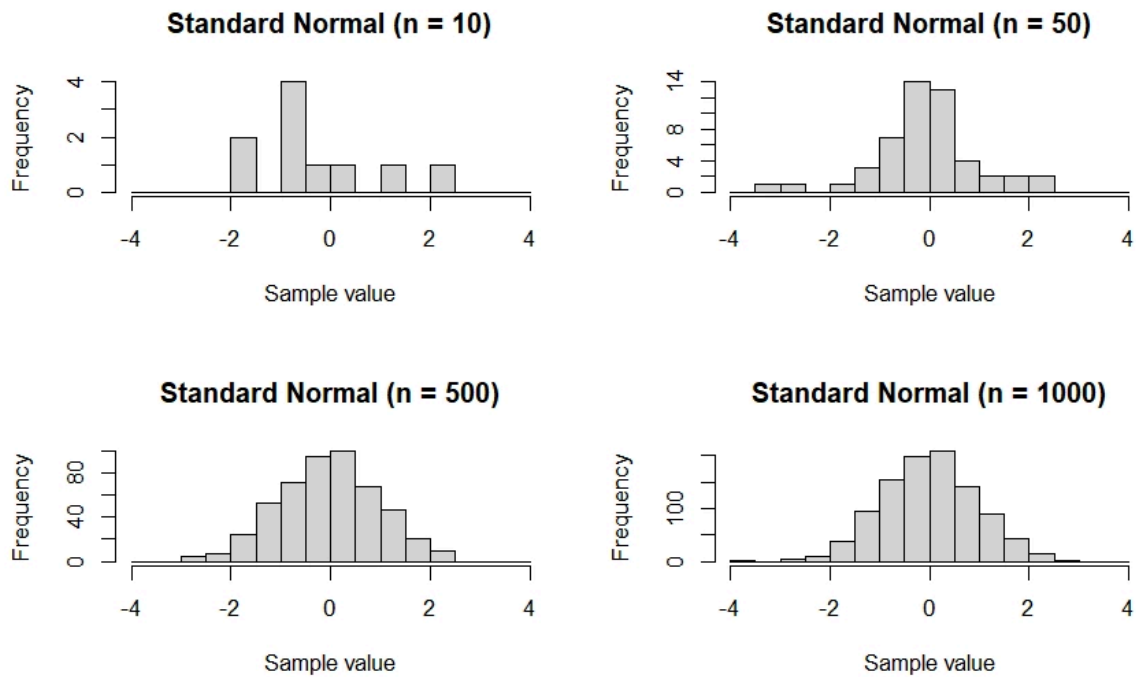


Figure: Random Samples drawn from $N(0,1)$ in different sizes

```
rdnum <- rnorm(10) # default: mean = 0, sd = 1
rdnum_2 <- rnorm(n = 10, mean = 60, sd = 15)
```

Control Flow

- By default, a program executes instructions one after another. **Control flow** allows the program to change this order by repeating actions with **for** loop or by making decisions with **if-else** statements

for loop

- A for loop repeats the same task over and over for each item in a list or for a fixed number of times

```
for (i in 1:5){ print(i) } # output [1] 1 [1] 2 [1] 3 [1] 4 [1] 5
```

if-else

- An **if-else** statement tells the computer what to do based on whether a condition is **TRUE** or **FALSE**

```
# syntax if(a < b){ # do work 1 }else if(a == b){ # do work 2 }else{ # do work 3 } #
example for (i in 55:65){ if(i < 60){ print("Fail:") }else if(i == 60){ print("On the
threshold!") }else{ print("Pass:") } }
```

replicate()

- The `replicate()` function is another way to repeat an expression multiple times. It stores the results in a vector, matrix, or list

```
rep_value <- replicate(n = 3, "Hello!") rep_sample <- replicate(n = 4, rnorm(2, mean =
0, sd = 1)) # output [1] "Hello!" "Hello!" "Hello!" [,1] [,2] [,3] [,4] [1,] -0.102854
1.44482927 0.5796968 1.2482826 [2,] 1.159166 -0.03620607 -0.7763544 0.7197338 # the
output of a single trial is stored in columns
```

Define Your Functions!

- Besides learning how to use functions in packages, we can also define our own functions with `function()`

```
# syntax function_name <- function(param1, param2, ...){ # do something return(output) }
# example add <- function(a, b){ return(a+b) } add(50, 100) # 150
```

- There are several reasons to define your own functions
 - Avoid copy-paste the same code for repeated use
 - Customize the parameters
 - You can save functions in a `R` script (e.g., `myfunctions.R`) and import it when you need it!

Assign Column Names

```
df <- data.frame(a = 1:3, b = 4:6) # output a b 1 1 4 2 2 5 3 3 6 colnames(df) <-
c("height", "weight") # output height weight 1 1 4 2 2 5 3 3 6
```

Exercises

 W5 Exercises



由Print Notion導出